Charla de arquitectura

Discusión conceptual previa al arranque de un sistema para un estudio jurídico — stack PERN + Prisma, equipo de 7 (4 backend / 3 frontend). Referencia concreta: módulo **SUPPLIERS** del proyecto Tornería Castro (NestJS).

PREGUNTA — mensaje inicial

Quiero charlar sobre arquitectura antes de arrancar un proyecto nuevo (sistema para un estudio jurídico, PERN + Prisma, equipo de 7: 4 backend / 3 frontend).

Contexto: en este proyecto (Torneria Castro, NestJS) usamos partición por dominio a nivel de estructura general, y dentro de cada módulo de entidad aplicamos arquitectura hexagonal. Como referencia concreta, mirá cómo está armado el módulo SUPPLIERS.

Quiero debatir a nivel conceptual (todavía no busco código ni estructura de carpetas final), estas dudas:

- 1. ¿Lo que hacemos en SUPPLIERS es realmente hexagonal "puro" o es una variante? ¿Cómo se llamaría bien este estilo (partición por dominio + hexagonal interno)?
- 2. ¿Un monolito o monolito modular puede tener hexagonal por dentro? ¿Cambia algo respecto a lo que ya hacemos acá?
- 3. En NestJS, el módulo/controller nos resuelve solo el enrutamiento hacia cada módulo de entidad. En PERN con Express, ¿qué tan complejo es replicar algo similar? Ayudame a decidir entre Express "pelado" armando esa capa de enrutamiento a mano, o alguna alternativa (Fastify con plugins, NestJS también sobre Prisma, u otra) — ventajas y desventajas de cada una para nuestro caso.
- 4. ¿Cuál es el criterio correcto para la partición por dominio: un módulo por entidad, o un módulo por grupo de entidades altamente cohesivas? ¿Esto cambia si usamos hexagonal + PERN en vez de NestJS?

Quiero que sea una charla, con ventajas/desventajas de cada opción, no una implementación todavía.

RESPUESTA

Antes de opinar dejame mirar cómo está armado SUPPLIERS acá para hablar sobre algo concreto, no en abstracto.

Ya miré el módulo (`backend/src/suppliers/`: `domain/` con `Supplier` entity + `Cuit` VO + errores + puertos, `application/` con use cases inyectando el puerto, `infrastructure/` con `InMemorySupplierRepository`, `http/` con controller + DTOs + filter + mapper, y `suppliers.module.ts` wireando todo). Está bien armado, así que puedo hablar sobre algo concreto. Vamos por las cuatro.

---

1. ¿Es hexagonal "puro"? ¿Cómo se llama el estilo?

Sí es hexagonal, en el sentido fuerte — no una versión aguada:

- El dominio (`Supplier`, `Cuit`, `SupplierType`, errores) no depende de nada externo, tiene invariantes propias, y expone factorías (`create` / `reconstruct`) en vez de dejarse construir crudo.
- Los **puertos** (`SupplierRepository`) los declara el dominio, no la infra. Esa es la línea que separa hexagonal de "solo tener una carpeta llamada `domain`".
- Los casos de uso (`CreateSupplier`, etc.) orquestan el puerto, no saben qué hay del otro lado.
- Hay adaptador **entrante** (`http/`, el controller convierte HTTP → llamada a use case) y adaptador **saliente** (`infrastructure/in-memory-supplier.repository.ts`, mañana el de Prisma). Es la definición del patrón.
- La dependencia va siempre hacia adentro: http → application → domain; infrastructure → domain (implementa el puerto). Domain no conoce a nadie.

Dónde no es 100% purista (y por qué está bien igual):

- Los use cases llevan `@Injectable()` y el puerto es `abstract class` para servir como token de DI. Un purista diría "la aplicación no debería saber que existe Nest". En la práctica, si un día quisieran sacar Nest, el cambio es reemplazar el decorator y el token — el diseño no se rompe. Es una atadura contenida.
- No hay carpeta `adapters/in/` y `adapters/out/` explícita; `http/` e `infrastructure/` cumplen ese rol con otro nombre. Nomenclatura, no estructura.

Cómo llamarlo bien: lo más honesto y descriptivo es **"monolito modular con hexagonal por módulo"** (o *"modular monolith, hexagonal-per-module"*). Cada módulo termina siendo un **bounded context** en términos DDD. No es "vertical slice architecture" — VSA agrupa por *caso de uso*, no por entidad/aggregate. Y no es "clean architecture" (Uncle Bob) aunque se parezca; clean pide separar `entities` de `use-cases` en capas concéntricas, con más ceremonia. Lo suyo es más liviano y directo: **DDD-lite + Ports & Adapters, un contexto por módulo**.

---

2. Monolito / monolito modular con hexagonal por dentro

Sí, es la combinación más popular hoy y es exactamente lo que están haciendo. Diría más: **hexagonal sin fronteras de módulo termina en una sola bola de barro con carpetitas prolijas**. Hexagonal necesita un límite claro alrededor de cada dominio; el "módulo" es ese límite.

Ventajas frente a otras combinaciones:

- **Un solo deploy**, tooling simple, transacciones DB cruzadas todavía posibles si las necesitás (algo que los microservicios te complican).
- Cada módulo es **extraíble** el día que un contexto se vuelva un servicio aparte: como el dominio no depende de nada, sacás `expedientes/` completo y le ponés su propio HTTP.
- Testéas por módulo con adaptadores fake (como ya hacen con `InMemorySupplierRepository` que reusan como fixture — muy buena movida).

Qué cambia respecto de lo que ya hacen: casi nada de estructura interna. Lo que aparece cuando el proyecto crece y **hay comunicación entre módulos** es una disciplina extra:

- Si `honorarios/` necesita datos de `expedientes/`, no puede importar `ExpedienteRepository` directo. Debería consumir una **API pública** de expedientes (un puerto entrante que expone

expedientes, o eventos de dominio). Esa disciplina hoy en Torneria casi no aparece porque cada módulo es autocontenido.

- Regla de dedo: **imports cruzados solo hacia la API pública del otro módulo** (típicamente un `index.ts` de barrel del módulo que exporta solo lo que es contrato). Un lint rule (`eslint-plugin-boundaries` o similar) lo hace cumplir sin depender de vigilancia humana.

En un estudio jurídico esto sí va a pasar: expedientes ↔ clientes ↔ honorarios ↔ agenda se cruzan todo el tiempo.

---

3. Enrutamiento tipo Nest en Express: qué opciones tenés

Lo que Nest resuelve gratis y hay que replicar:

- **Enrutamiento declarativo por módulo/controller** (agrupar rutas por clase).
- **DI container** con tokens (clave para el patrón puerto/adaptador que ya usan).
- **Global validation pipe** (DTOs con validación declarativa).
- **Exception filters** globales (domain error → status HTTP).
- **Módulos como unidades de composición**.

Nada de eso es imposible en Express, pero hay que construirlo. Opciones ordenadas por cuánto tenés que armar:

A) Express "pelado" + patrones a mano

- Un `router` per module, un "composition root" por módulo que cablea a mano (`new CreateSupplier(new PrismaSupplierRepository(prisma))`), Zod para validación de request, un error handler global que mapea `DomainError` → status.
- **Pros**: cero magia, cualquier junior lo lee, control total, ecosistema masivo, curva bajísima.
- **Contras**: sin convención escrita, cada backender va a inventar su propio patrón — con 4 backends te termina saliendo un módulo prolijo y tres que no. El "wiring" a mano es tolerable con 5 módulos, molesto con 15. Sin DI container, testear reemplazando dependencias es a mano.

B) Express + DI container liviano (awilix o tsyringe) + Zod

- Lo mismo que A pero el wiring lo hace un container. Cada módulo registra sus providers, la composición se ve declarativa.
- **Pros**: te acerca mucho a Nest sin cargar todo el framework; hexagonal encaja natural; la mayoría del boilerplate desaparece; testeabilidad muy buena.
- **Contras**: 2–3 días de armar el "esqueleto de referencia" y **escribirlo como convención** para que el equipo lo respete. Sin ese doc + un ejemplo (tipo el `SuppliersModule` de acá), degrada rápido.

C) Fastify + plugins

- **Pros**: 2–3× más rápido que Express, validación de request/response por JSON Schema o Zod integrada con tipado end-to-end (`fastify-type-provider-zod`), sistema de plugins que se

comporta como módulos, comunidad sana.

- **Contras**: DI sigue afuera (mismo `awilix`/`tsyringe`), comunidad más chica que Express, alguna dependencia npm menos común. Para juniors que nunca vieron Express es indistinto; para seniors acostumbrados a Express es un cambio menor.

D) NestJS + Prisma

- Es lo que ya hacen. Todo lo que replicarías en A/B/C viene resuelto: `@Module`, DI con tokens (usan `abstract class` como token, exactamente el patrón que necesitan), `ValidationPipe`, exception filters, testing con `Test.createTestingModule`.
- **Pros**: el equipo ya sabe, cero fricción de bootstrap, hexagonal encaja limpio (como ya se ve en Suppliers), Prisma se integra con `PrismaService` inyectable en dos líneas.
- **Contras**: framework opinionado, `reflect-metadata` + decoradores (magia moderada), atadura leve en use cases. Si "PERN" es un requisito duro (curso, empleador, consigna), tirás Nest a la basura.

Mi lectura para su caso

(7 personas, 4 back, arrancando de cero, quieren hexagonal por módulo):

- Si **"PERN" no es innegociable**: **Nest + Prisma**. Es reusar lo que ya saben, la arquitectura queda idéntica a Torneria, y ahorran 2–3 semanas de armar el esqueleto.
- Si **PERN es innegociable** (curso, gusto, aprendizaje): **Fastify +** `awilix` **+ Zod**, no Express pelado. Le ganás performance a Express, tenés validación tipada, y el DI container te resuelve el problema real (wiring). Costo: 1 semana de esqueleto y convención.
- **Express pelado solo si el objetivo pedagógico es armar el andamiaje a mano**. Con 7 personas y sin escribir convención estricta, tiende a heterogéneo.

Un detalle: la parte "R" de PERN (React) no cambia si en el backend eligen Nest o Express, y la "P" (Postgres) tampoco. Nadie se enoja porque la "E" sea Fastify o Nest — la sigla en la vida real es negociable.

---

4. Criterio de partición: entidad vs grupo cohesivo

En DDD canónico la unidad no es la entidad, es el aggregate (raíz + entidades hijas + VOs que viven y mueren juntos). Un módulo = un aggregate = un bounded context chico. Ejemplo: `Order` con `OrderLine` es **un** módulo `orders/`, no dos, porque `OrderLine` no tiene sentido sin su `Order`.

Criterio pragmático que uso:

- ¿Estas dos entidades tienen **ciclos de vida independientes**? (¿puede existir A sin B?) → módulos separados.
- ¿Tienen **casos de uso independientes**? (¿alguien crea/lista/actualiza una sin tocar la otra?) → módulos separados.
- ¿Comparten **invariantes que se validan juntas**? → mismo módulo (mismo aggregate).
- Si dudás, empezá **más grueso** y partí después. Mergear dos módulos es doloroso; partir uno es fácil si el dominio ya está limpio.

En el módulo Suppliers funciona "un módulo por entidad" porque Supplier no tiene hijos que le pertenezcan; `SupplierType` es un enum, no una entidad. Perfecto módulo-por-aggregate que casualmente coincide con módulo-por-entidad.

Para un estudio jurídico, "un módulo por entidad" te va a llevar a fragmentar mal. Los módulos que probablemente aparezcan son áreas de contexto, no tablas:

- `expedientes/` — el aggregate incluye el expediente, sus partes, sus movimientos, sus plazos. Adentro son varias tablas, un solo módulo.
- `clientes/` — persona física/jurídica + contactos + representantes legales.
- `honorarios/` — facturas, cobros, pactos.
- `agenda/` — audiencias, vencimientos, alertas.
- `documentos/` — plantillas, generados, adjuntos.
- `usuarios/` (o `iam/`) — cuentas, roles, permisos.

Cada uno probablemente tenga 3–6 entidades adentro, y ninguna de ellas justifica un módulo propio.

¿Cambia el criterio con hexagonal + PERN vs Nest? Conceptualmente no. Cambia la **fricción práctica**:

- En Nest, agregar un módulo cuesta un `XxxModule` — es barato, tentador, y termina en sobre-fragmentación por entidad si el equipo no tiene disciplina.
- En Express/Fastify pelado, agregar un módulo cuesta router + composition root + tests wireados a mano — es más caro, y el equipo va a tender natural a módulos más gruesos por bounded context.

Ese costo de "crear un módulo" es un incentivo real de diseño. En Torneria, ver `suppliers/`, `products/`, `clients/` como módulos separados tiene sentido porque no comparten aggregate. En un estudio jurídico, ver `expedientes/`, `partes/`, `movimientos/`, `plazos/` como módulos separados sería un error — son un solo bounded context.

Si querés que profundicemos en alguno (por ejemplo: cómo modelarías la comunicación entre `expedientes` y `honorarios` sin acoplarlos, o el trade-off concreto Fastify+awilix vs Nest en detalle), me decís.

PREGUNTA — seguimiento

Bien perfecto y es recomendable usar hexagonal para este caso o se puede usar monolitico modular con otro tipo de estructura interna? que nos conviene?

RESPUESTA

Buena pregunta y es donde muchos equipos se traban: **hexagonal no es gratis**, y aplicarlo a todo por defecto termina siendo más ceremonia que valor. Te tiro el panorama y después una recomendación concreta.

Cuándo hexagonal realmente paga

Hexagonal cobra su precio (más carpetas, más archivos, más wiring, más objetos: entity + VO + puerto + adaptador + mapper) y devuelve valor cuando:

- **El dominio tiene reglas propias no triviales** — invariantes, transiciones de estado, cálculos, validaciones cruzadas. Ejemplo típico: un expediente que no puede pasar de "en trámite" a "archivado" si tiene plazos abiertos, un honorario que se calcula sobre un pacto con reglas por tipo de fuero, etc.
- **La persistencia puede cambiar** o querés testear sin DB (adaptador in-memory como el que ya usan en Suppliers).
- **Vas a tener más de un adaptador entrante** (HTTP + jobs + CLI + webhooks).
- **El equipo es mediano/grande** y necesitás una convención estricta que sobreviva a la rotación.
- **La vida útil del sistema es larga** (5+ años). El costo inicial se amortiza.

Donde NO paga:

- Módulos de puro CRUD sin reglas (catálogos, ABMs de referencia). Ahí hexagonal termina siendo entity → puerto → adaptador → controller → DTO → mapper para hacer un `INSERT`.
- Prototipos, MVPs cortos, scripts.
- Equipos muy chicos donde la ceremonia sobra.

Alternativas dentro de un monolito modular

Capas clásicas (N-tier / "service layer") — `controller → service → repository`, sin abstract ports, sin entities ricas. El "modelo" son tipos de Prisma directamente. La lógica vive en services que reciben Prisma o repositorios concretos.

- **Pros**: mucho menos código, arranque rápido, cualquier junior lo lee, es el pattern por defecto de la comunidad Node.
- **Contras**: sin puertos, testear un service implica mockear Prisma; la lógica de negocio termina desparramada entre service y controller; con el tiempo aparecen los "fat services" de 800 líneas.

Vertical Slice Architecture (VSA) — organizás por **caso de uso**, no por entidad. `features/crear-expediente/`, `features/cerrar-expediente/`, cada carpeta con handler + DTO + validación + query, todo junto. Nada compartido salvo el dominio.

- **Pros**: acoplamiento por feature altísimo, cambios localizados, cada slice se lee sin saltar entre 6 archivos, muy amigable para equipos que se dividen trabajo por historia.
- **Contras**: duplica código (dos features similares repiten cosas), cuesta más cuando aparece un aggregate rico compartido entre features, y requiere disciplina para no re-fragmentar (la tentación es hacer "helpers compartidos" y volvés al problema de siempre).

Active Record / anemic + Prisma directo — usás los modelos de Prisma como entidades, la validación vive en el DTO (Zod/class-validator), y hay services que orquestan.

- **Pros**: velocidad máxima, cero mapping.
- **Contras**: el dominio vive escrito en el schema de Prisma. Las invariantes se replican en cada service. Al año, cambiar una regla implica buscar en 15 lugares.

CQRS (sin event sourcing) — separás modelo de lectura de modelo de escritura. Los reads van directo a Prisma con projections; los writes pasan por el aggregate.

- **Pros**: las listas/reportes son SQL crudo optimizado; los writes conservan el rigor.
- **Contras**: doble modelo, más ceremonia. Solo vale la pena cuando hay reportería pesada o lecturas muy asimétricas de las escrituras.

Para un estudio jurídico, concretamente

El dominio de un estudio **no es CRUD parejo**:

- **Expedientes, plazos, honorarios, agenda** tienen reglas serias — estados legales, cómputo de plazos hábiles, cálculo de honorarios con arancel, notificaciones automáticas cuando vence algo. **Ahí hexagonal paga bien**.
- **Clientes, contactos, catálogos (tipos de fuero, jurisdicciones, plantillas)** son mayormente CRUD. **Ahí hexagonal es sobre-ingeniería**.

Mi recomendación honesta: **hexagonal selectivo, no hexagonal universal**.

Estrategia práctica

- **Un solo estilo base para todo el proyecto** (por consistencia y curva del equipo): capa clásica con controller → service → repository, con Prisma inyectado en repositories. Convención escrita, un módulo de referencia armado.
- **Módulos "ricos" suben a hexagonal** cuando su dominio lo justifica: entity + VO + ports + adapters. El resto se queda en la capa clásica.
- La frontera entre módulos sigue siendo estricta (API pública por módulo, no imports cruzados a lo profundo) — eso es lo que hace al monolito **modular**, y no depende del estilo interno del módulo.

Esto te evita el escenario típico: equipo de 7 forzado a hacer hexagonal para todo, terminan haciéndolo mal en los módulos CRUD (adaptadores in-memory que nadie usa, mappers triviales, puertos con un solo método) y bien solo donde importa.

Contra-argumento válido (por qué elegir hexagonal para todo igual)

- **Uniformidad**: un solo patrón, un solo template mental, un solo estilo de test. Con 7 personas y niveles distintos, la uniformidad tiene valor real.
- **No sabés hoy cuál módulo se va a volver "rico"**. Si `clientes/` arranca CRUD y en 6 meses aparecen reglas de conflicto de intereses, migrar de capa clásica a hexagonal es más caro que haberlo hecho desde el día uno.
- **El equipo ya sabe hacerlo** (por Torneria). Si mezclás estilos, cada módulo se decide caso a caso y ese debate consume energía.

Si el equipo es sólido y el proyecto se piensa a largo plazo, **hexagonal parejo es la opción segura**. Si querés optimizar velocidad de entrega y aceptar refactor futuro, **híbrido**. No hay una respuesta única.

Lo que sí es común a todas las opciones

Independientemente del estilo interno, hay tres cosas que **no** son negociables si van monolito modular:

- **Frontera modular estricta**: un módulo no importa nada de otro que no sea su API pública (un `index.ts` que exporta contratos, no repositorios ni entidades sueltas). Lint rule que lo hace cumplir.
- **Cada módulo dueño de sus tablas**: ninguna otra parte del sistema toca la DB de otro módulo. Se accede vía API pública.
- **Un solo estilo de validación, un solo estilo de manejo de errores, un solo estilo de tests** — la consistencia entre módulos vale más que la elegancia dentro de uno.

Si tuviera que darte una sola frase: **para un estudio jurídico con equipo de 7 y visión a 3+ años, iría con hexagonal parejo (como Torneria) para no pelear con la heterogeneidad, aceptando el "impuesto de ceremonia" en los módulos CRUD como precio de la uniformidad**. Si el equipo tuviera más seniority pareja, iría híbrido y me ahorraría ese impuesto.

¿Querés que profundicemos en cómo decidís caso por caso qué módulo va rico y cuál no? ¿O querés que hablemos del frontend, que es la parte donde estas decisiones suelen tener menos guía?

PREGUNTA — detalle sobre frontera modular estricta

Explicame esto:

"Frontera modular estricta: un módulo no importa nada de otro que no sea su API pública (un index.ts que exporta contratos, no repositorios ni entidades sueltas). Lint rule que lo hace cumplir."

RESPUESTA

Es una idea corta pero con más miga de lo que parece. Te la explico en tres pasos: **el problema**, **cómo se estructura la solución**, y **cómo se hace cumplir automáticamente**.

El problema que resuelve

En un monolito modular, "módulos" es más una **promesa de disciplina** que una barrera técnica. Nada te impide, desde `honorarios/`, hacer:

```
import { ClienteRepository } from '../clientes/infrastructure/prisma-cliente.repository';
import { Cliente } from '../clientes/domain/cliente';
```

TypeScript lo compila feliz. Todo funciona. Y acabás de romper el módulo sin darte cuenta:

- Ahora `clientes/` **no puede refactorizar** su repositorio ni su entidad sin romper `honorarios/`. La libertad interna del módulo desapareció.
- La entidad `Cliente` viaja fuera de su módulo con **todos sus métodos** (`activate()`, `deactivate()`, `update()`, etc.). Cualquiera puede llamarlos y romper invariantes que solo los use cases de `clientes/` sabían mantener.
- Ese `Cliente` importado empieza a aparecer en firmas de funciones de `honorarios/`, después en `agenda/`, y de pronto el "módulo" `clientes/` es una carpeta llena de código que **todo el sistema conoce por dentro**. Volviste al mono-módulo con carpetitas.
- Si mañana quieren extraer `clientes/` a un microservicio, no pueden — la mitad del sistema depende de sus internals.

El síntoma clásico: la carpeta `clientes/` tiene 40 archivos y **cualquiera de los 40 es importado desde otro módulo**. Ya no hay módulo, hay namespace.

Cómo se estructura la solución

Cada módulo tiene **un único punto de entrada** (un archivo tipo `index.ts` en la raíz del módulo, o una carpeta `public/`) que decide qué es contrato y qué es interno. Todo lo demás es privado por convención.

Lo que **sí** exporta el barrel — el "API pública" del módulo:

- **Interfaces / puertos entrantes**: `ClientesApi` con métodos como `getClienteBasicInfo(id)`, `existsCliente(id)`. Son los servicios que el módulo ofrece al resto.
- **DTOs de solo lectura**: `ClienteView`, `ClienteBasicInfo` — la forma pública que expone el módulo. No es la entity, es una proyección plana.

- **IDs y tipos compartidos**: `ClienteId`, enums como `TipoCliente`.
- **Eventos de dominio**: `ClienteCreado`, `ClienteDadoDeBaja` — si el módulo publica cosas para que otros escuchen.
- **Errores públicos**: `ClienteNoEncontradoError` (si otros módulos necesitan atraparlo).

Lo que **no** exporta:

- La entity `Cliente` (con sus métodos mutadores).
- El repositorio (`ClienteRepository`), ni el puerto ni el adaptador.
- Los use cases (`CreateCliente`, `UpdateCliente`) directamente como clases.
- Cualquier detalle de infra: modelos Prisma, mappers, DTOs internos.

En términos concretos, con la estructura de Torneria, la raíz de `clientes/` tendría:

```
clientes/
■■■ domain/            (interno)
■■■ application/       (interno)
■■■ infrastructure/    (interno)
■■■ http/              (interno)
■■■ clientes.module.ts (interno — wiring)
■■■ index.ts           ← la única puerta
```

Y `index.ts` exporta algo así:

```
export type { ClientesApi } from './public/clientes.api';
export type { ClienteView, ClienteId } from './public/cliente.view';
export { ClienteNoEncontradoError } from './public/errors';
export { ClienteCreadoEvent } from './public/events';
// nada mas
```

Cuando `honorarios/` necesita saber quién es el cliente de un expediente, no importa la entity: pide una `ClientesApi` por DI y llama `getClienteBasicInfo(id)`, que le devuelve un `ClienteView` — un objeto plano, sin métodos. Si `clientes/` cambia su entity mañana, `honorarios/` no se entera.

El detalle importante: `ClientesApi` es una **interfaz**. La implementación concreta la registra `clientes/` en el composition root (con Nest, en su `ClientesModule` con `providers: [{ provide: ClientesApi, useClass: ClientesApiImpl }]` y `exports: [ClientesApi]`). `honorarios/` la consume vía DI. Nunca importa la clase concreta.

Comunicación entre módulos, dos variantes

Este patrón te da dos formas naturales de comunicar módulos:

- **Sincrónica (query)**: módulo A necesita saber algo de B ahora → interfaz `BApi` en el barrel de B, inyectada en A. Simple, transaccional si están en el mismo proceso.
- **Asincrónica (evento)**: módulo A hace algo y B quiere reaccionar → A publica un evento en el barrel, B se suscribe. Ninguno depende del otro; podés reemplazar B sin tocar A.

Regla de dedo: **queries para leer, eventos para reaccionar**. Si `honorarios` necesita el nombre del cliente para armar una factura → query. Si `notificaciones` quiere mandar un mail cuando se crea un cliente → evento.

Cómo se hace cumplir con lint (la parte que evita depender de la buena voluntad)

Sin regla automática, el patrón dura tres meses. Cualquier dev apurado hace un import "solo por esta vez" y ya está.

Herramientas típicas en Node/TS:

- `eslint-plugin-boundaries` — declarás tipos de elementos (por ejemplo "modulo") y qué puede importar qué. La regla más común: cualquier archivo dentro de `src/modules/*/*` solo puede importar de otro módulo a través de `src/modules/*/index.ts`. Los imports profundos se rechazan.
- `eslint-plugin-import` **con** `no-restricted-paths` — más manual pero suficiente si sos pocos módulos. Definís "desde estas carpetas no se puede importar de estas otras carpetas, excepto por estos archivos".
- **Nx workspace** — si el proyecto crece a un monorepo, Nx tiene esto nativo con "tags" y `enforce-module-boundaries`. Cada librería declara tags y qué puede depender de qué.
- **TypeScript project references +** `paths` — camino más nuclear: cada módulo es un subproyecto TS separado, y `paths` en `tsconfig` solo expone su barrel. Muy estricto, más overhead de tooling.

La regla concreta, en palabras: **"desde** `modules/A/**` **no se puede importar** `modules/B/**` **salvo** `modules/B/index.ts`**"**. Cualquier import profundo (por ejemplo `modules/clientes/domain/cliente`) da error de lint y no mergea.

Con eso, la disciplina deja de depender de que el revisor de PR se dé cuenta. El linter la impone.

Qué te compra todo esto, en una línea

Cada módulo se vuelve **refactorable por dentro sin coordinar con nadie**, y **reemplazable por fuera** (podés meter una implementación fake, mockear en tests, o extraerlo a un servicio). La frontera es lo que hace al modular monolith **modular** en serio y no solo "un monolito con carpetas".

Y el costo es mínimo: un archivo barrel por módulo y una regla de ESLint. Comparado con el dolor de desacoplar módulos que se enredaron durante dos años, es baratísimo.

¿Querés que veamos cómo se ve un ejemplo concreto de `clientes/` exponiendo su API pública y `honorarios/` consumiéndola, con la parte de eventos incluida? Sigo sin escribir código de producción, es para bajarlo a algo tangible antes de decidir.